United States Patent	12388845
Kind Code	B2
Date of Patent	August 12, 2025
Inventor(s)	Ditchburn; Martyn

Zero trust lensing of cloud-based traffic

Abstract

Systems and methods for zero trust lensing of cloud-based traffic. Various embodiments include intercepting a plurality of requests from one or more users in a cloud-based system, wherein the requests are for connectivity to one or more destination workloads; determining a request of the plurality of requests requires lensing; identifying an appropriate lens for rendering the request; and initiating and utilizing a lens workload for processing a session associated with the request, wherein the session is processed in-line between the user and the workload destination without the user nor the workload destination being aware of the lens.

Inventors: **Ditchburn; Martyn (London, GB)**

Applicant: **Zscaler, Inc.** (San Jose, CA)

Family ID: **91962854**

Assignee: **Zscaler, Inc. (San Jose, CA)**

Appl. No.: **18/160462**

Filed: **January 27, 2023**

Prior Publication Data

Document Identifier	Publication Date
US 20240259398 A1	Aug. 01, 2024

Publication Classification

Int. Cl.: **H04L9/40** (20220101); **G06F9/455** (20180101)

U.S. Cl.:

CPC **H04L63/1416** (20130101); **G06F9/45558** (20130101); G06F2009/45587 (20130101)

Field of Classification Search

CPC: H04L (63/1416); G06F (9/45558); G06F (2009/45587)

USPC: 726/23; 709/223; 709/224

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
12041059	12/2023	Simic	N/A	H04L 63/083
2022/0182363	12/2021	Bharti	N/A	H04L 67/306
2023/0114821	12/2022	Thomas	726/23	H04L 63/1433

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
114461610	12/2021	CN	N/A

Primary Examiner: Wang; Liang Che A

Attorney, Agent or Firm: Baratta Law PLLC

Background/Summary

FIELD OF THE DISCLOSURE

(1) The present disclosure relates generally to networking and computing. More particularly, the present disclosure relates to systems and methods for zero trust lensing of cloud-based traffic.

BACKGROUND OF THE DISCLOSURE

(2) The traditional view of an enterprise network (i.e., corporate, private, industrial, operational, etc.) included a well-defined perimeter defended by various appliances (e.g., firewalls, intrusion prevention, advanced threat detection, etc.). In this traditional view, mobile users utilize a Virtual Private Network (VPN), etc. and have their traffic backhauled into the well-defined perimeter. This worked when mobile users represented a small fraction of the users, i.e., most users were within the well-defined perimeter. However, this is no longer the case—the definition of the workplace is no longer confined to within the well-defined perimeter, and with applications moving to the cloud, the perimeter has extended to the Internet. This results in an increased risk for the enterprise data residing on unsecured and unmanaged devices as well as the security risks in access to the Internet. Cloud-based security solutions have emerged, such as Zscaler Internet Access (ZIA) and Zscaler Private Access (ZPA), available from Zscaler, Inc., the applicant and assignee of the present application.

(3) ZPA is a cloud service that provides seamless, zero trust access to private applications running on the public cloud, within the data center, within an enterprise network, etc. As described herein, ZPA is referred to as zero trust access to private applications or simply a zero trust access service. Here, applications are never exposed to the Internet, making them completely invisible to unauthorized users. The service enables the applications to connect to users via inside-out connectivity versus extending the network to them. Users are never placed on the network. This Zero Trust Network Access (ZTNA) approach supports both managed and unmanaged devices and any private application (not just web apps).

BRIEF SUMMARY OF THE DISCLOSURE

(4) In an embodiment, the present disclosure includes a method with steps, a cloud-based system configured to implement the steps, and a non-transitory computer-readable medium storing computer-executable instructions for causing performance of the steps. The steps include intercepting a plurality of requests from one or more users in a cloud-based system, wherein the requests are for connectivity to one or more destination workloads; determining a request of the plurality of requests requires lensing; identifying an appropriate lens for rendering the request; and initiating and utilizing a lens workload for processing a session associated with the request, wherein the session is processed in-line between the user and the workload destination without the user nor the workload destination being aware of the lens.

(5) The steps can further include wherein the lens is adapted to provide one of a specific function associated with a specific application and a full Virtual Desktop Infrastructure (VDI) experience. The steps are performed within a cloud-based system utilizing a zero trust architecture. The appropriate lens is selected from a repository including a plurality of lenses, and wherein the plurality of lenses includes any of Application Programing Interfaces (APIs), containers, libraries, and virtual machine hosted services. The repository of lenses includes any of stock application lenses, custom application lenses, lenses from a vendor application lens marketplace, and Virtual Desktop Infrastructure (VDI) lenses. The vendor application lens marketplace is an extension of the lens repository, and wherein the vendor application lens marketplace includes a collection of vendor maintained and provisioned lenses. Lens workloads are created and utilized for the life of a session and subsequently terminated at the end of each session. The request can be passed to a private access connector responsive to determining that no lens is required.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) The present disclosure is illustrated and described herein with reference to the various drawings, in which like reference numbers are used to denote like system components/method steps, as appropriate, and in which:

(2) FIG. **1**A is a network diagram of a cloud-based system offering security as a service.

(3) FIG. **1**B is a logical diagram of the cloud-based system operating as a zero-trust platform.

(4) FIG. **1**C is a logical diagram illustrating zero trust policies with the cloud-based system and a comparison with the conventional firewall-based approach.

(5) FIG. **2** is a network diagram of an example implementation of the cloud-based system.

(6) FIG. **3** is a block diagram of a server, which may be used in the cloud-based system, in other systems, or standalone.

(7) FIG. **4** is a block diagram of a user device, which may be used with the cloud-based system or the like.

(8) FIG. **5** is a network diagram of a Zero Trust Network Access (ZTNA) application utilizing the cloud-based system.

(9) FIG. **6** is a network diagram of the cloud-based system in an application of digital experience monitoring.

(10) FIG. **7** is a flow diagram of an embodiment for zero trust lensing of the present disclosure.

(11) FIG. **8** is a flow diagram of an embodiment of the present zero trust lens architecture.

(12) FIG. **9** is a flow diagram of an embodiment of zero trust lensing utilizing various side containers.

(13) FIG. **10** is a flow diagram of an embodiment of the present zero trust lens workflow logic.

(14) FIG. **11** is a flow chart of a process for zero trust lensing of cloud-based traffic.

DETAILED DESCRIPTION OF THE DISCLOSURE

(15) The present disclosure relates to systems and methods for zero trust lensing of cloud-based traffic. Zero trust lensing is the method by which client-initiated traffic is intercepted and filtered through an alternate viewing mechanism prior to reaching its destination. In various embodiments, neither the source nor destination applications associated with the traffic are aware of the translation and sit independent and unaltered during the exchange.

Example Cloud-Based System Architecture

(16) FIG. **1**A is a network diagram of a cloud-based system **100** offering security as a service. Specifically, the cloud-based system **100** can offer a Secure Internet and Web Gateway as a service to various users **102**, as well as other cloud services. In this manner, the cloud-based system **100** is located between the users **102** and the Internet as well as any cloud services **106** (or applications) accessed by the users **102**. As such, the cloud-based system **100** provides inline monitoring inspecting traffic between the users **102**, the Internet **104**, and the cloud services **106**, including Secure Sockets Layer (SSL) traffic. The cloud-based system **100** can offer access control, threat prevention, data protection, etc. The access control can include a cloud-based firewall, cloud-based intrusion detection, Uniform Resource Locator (URL) filtering, bandwidth control, Domain Name System (DNS) filtering, etc. The threat prevention can include cloud-based intrusion prevention, protection against advanced threats (malware, spam, Cross-Site Scripting (XSS), phishing, etc.), cloud-based sandbox, antivirus, DNS security, etc. The data protection can include Data Loss Prevention (DLP), cloud application security such as via a Cloud Access Security Broker (CASB), file type control, etc.

(17) The cloud-based firewall can provide Deep Packet Inspection (DPI) and access controls across various ports and protocols as well as being application and user aware. The URL filtering can block, allow, or limit website access based on policy for a user, group of users, or entire organization, including specific destinations or categories of URLs (e.g., gambling, social media, etc.). The bandwidth control can enforce bandwidth policies and prioritize critical applications such as relative to recreational traffic. DNS filtering can control and block DNS requests against known and malicious destinations.

(18) The cloud-based intrusion prevention and advanced threat protection can deliver full threat protection against malicious content such as browser exploits, scripts, identified botnets and malware callbacks, etc. The cloud-based sandbox can block zero-day exploits (just identified) by analyzing unknown files for malicious behavior. Advantageously, the cloud-based system **100** is multi-tenant and can service a large volume of the users **102**. As such, newly discovered threats can be promulgated throughout the cloud-based system **100** for all tenants practically instantaneously. The antivirus protection can include antivirus, antispyware, antimalware, etc. protection for the users **102**, using signatures sourced and constantly updated. The DNS security can identify and route command-and-control connections to threat detection engines for full content inspection.

(19) The DLP can use standard and/or custom dictionaries to continuously monitor the users **102**, including compressed and/or SSL-encrypted traffic. Again, being in a cloud implementation, the cloud-based system **100** can scale this monitoring with near-zero latency on the users **102**. The cloud application security can include CASB functionality to discover and control user access to known and unknown cloud services **106**. The file type controls enable true file type control by the user, location, destination, etc. to determine which files are allowed or not.

(20) For illustration purposes, the users **102** of the cloud-based system **100** can include a mobile device **110**, a headquarters (HQ) **112** which can include or connect to a data center (DC) **114**, Internet of Things (IoT) devices **116**, a branch office/remote location **118**, etc., and each includes one or more user devices (an example user device **300** is illustrated in FIG. **5**). The devices **110**, **116**, and the locations **112**, **114**, **118** are shown for illustrative purposes, and those skilled in the art will recognize there are various access scenarios and other users **102** for the cloud-based system **100**, all of which are contemplated herein. The users **102** can be associated with a tenant, which may include an enterprise, a corporation, an organization, etc. That is, a tenant is a group of users

who share a common access with specific privileges to the cloud-based system **100**, a cloud service, etc. In an embodiment, the headquarters **112** can include an enterprise's network with resources in the data center **114**. The mobile device **110** can be a so-called road warrior, i.e., users that are off-site, on-the-road, etc. Those skilled in the art will recognize a user **102** has to use a corresponding user device **300** for accessing the cloud-based system **100** and the like, and the description herein may use the user **102** and/or the user device **300** interchangeably.

(21) Further, the cloud-based system **100** can be multi-tenant, with each tenant having its own users **102** and configuration, policy, rules, etc. One advantage of the multi-tenancy and a large volume of users is the zero-day/zero-hour protection in that a new vulnerability can be detected and then instantly remediated across the entire cloud-based system **100**. The same applies to policy, rule, configuration, etc. changes—they are instantly remediated across the entire cloud-based system **100**. As well, new features in the cloud-based system **100** can also be rolled up simultaneously across the user base, as opposed to selective and time-consuming upgrades on every device at the locations **112**, **114**, **118**, and the devices **110**, **116**.

(22) Logically, the cloud-based system **100** can be viewed as an overlay network between users (at the locations **112**, **114**, **118**, and the devices **110**, **116**) and the Internet **104** and the cloud services **106**. Previously, the IT deployment model included enterprise resources and applications stored within the data center **114** (i.e., physical devices) behind a firewall (perimeter), accessible by employees, partners, contractors, etc. on-site or remote via Virtual Private Networks (VPNs), etc. The cloud-based system **100** is replacing the conventional deployment model. The cloud-based system **100** can be used to implement these services in the cloud without requiring the physical devices and management thereof by enterprise IT administrators. As an ever-present overlay network, the cloud-based system **100** can provide the same functions as the physical devices and/or appliances regardless of geography or location of the users **102**, as well as independent of platform, operating system, network access technique, network access provider, etc.

(23) There are various techniques to forward traffic between the users **102** at the locations **112**, **114**, **118**, and via the devices **110**, **116**, and the cloud-based system **100**. Typically, the locations **112**, **114**, **118** can use tunneling where all traffic is forward through the cloud-based system **100**. For example, various tunneling protocols are contemplated, such as Generic Routing Encapsulation (GRE), Layer Two Tunneling Protocol (L2TP), Internet Protocol (IP) Security (IPsec), customized tunneling protocols, etc. The devices **110**, **116**, when not at one of the locations **112**, **114**, **118** can use a local application that forwards traffic, a proxy such as via a Proxy Auto-Config (PAC) file, and the like. An application of the local application is the application **350** described in detail herein as a connector application. A key aspect of the cloud-based system **100** is all traffic between the users **102** and the Internet **104** or the cloud services **106** is via the cloud-based system **100**. As such, the cloud-based system **100** has visibility to enable various functions, all of which are performed off the user device in the cloud.

(24) The cloud-based system **100** can also include a management system **120** for tenant access to provide global policy and configuration as well as real-time analytics. This enables IT administrators to have a unified view of user activity, threat intelligence, application usage, etc. For example, IT administrators can drill-down to a per-user level to understand events and correlate threats, to identify compromised devices, to have application visibility, and the like. The cloud-based system **100** can further include connectivity to an Identity Provider (IDP) **122** for authentication of the users **102** and to a Security Information and Event Management (SIEM) system **124** for event logging. The system **124** can provide alert and activity logs on a per-user **102** basis.

(25) Zero Trust

(26) FIG. **1**B is a logical diagram of the cloud-based system **100** operating as a zero-trust platform. Zero trust is a framework for securing organizations in the cloud and mobile world that asserts that no user or application should be trusted by default. Following a key zero trust principle, least-

privileged access, trust is established based on context (e.g., user identity and location, the security posture of the endpoint, the app or service being requested) with policy checks at each step, via the cloud-based system **100**. Zero trust is a cybersecurity strategy wherein security policy is applied based on context established through least-privileged access controls and strict user authentication—not assumed trust. A well-tuned zero trust architecture leads to simpler network infrastructure, a better user experience, and improved cyberthreat defense.

(27) Establishing a zero trust architecture requires visibility and control over the environment's users and traffic, including that which is encrypted; monitoring and verification of traffic between parts of the environment; and strong multifactor authentication (MFA) methods beyond passwords, such as biometrics or one-time codes. This is performed via the cloud-based system **100**. Critically, in a zero trust architecture, a resource's network location is not the biggest factor in its security posture anymore. Instead of rigid network segmentation, your data, workflows, services, and such are protected by software-defined microsegmentation, enabling you to keep them secure anywhere, whether in your data center or in distributed hybrid and multicloud environments.

(28) The core concept of zero trust is simple: assume everything is hostile by default. It is a major departure from the network security model built on the centralized data center and secure network perimeter. These network architectures rely on approved IP addresses, ports, and protocols to establish access controls and validate what's trusted inside the network, generally including anybody connecting via remote access VPN. In contrast, a zero trust approach treats all traffic, even if it is already inside the perimeter, as hostile. For example, workloads are blocked from communicating until they are validated by a set of attributes, such as a fingerprint or identity. Identity-based validation policies result in stronger security that travels with the workload wherever it communicates—in a public cloud, a hybrid environment, a container, or an on-premises network architecture.

(29) Because protection is environment-agnostic, zero trust secures applications and services even if they communicate across network environments, requiring no architectural changes or policy updates. Zero trust securely connects users, devices, and applications using business policies over any network, enabling safe digital transformation. Zero trust is about more than user identity, segmentation, and secure access. It is a strategy upon which to build a cybersecurity ecosystem.

(30) At its Core are Three Tenets:

(31) Terminate every connection: Technologies like firewalls use a “passthrough” approach, inspecting files as they are delivered. If a malicious file is detected, alerts are often too late. An effective zero trust solution terminates every connection to allow an inline proxy architecture to inspect all traffic, including encrypted traffic, in real time—before it reaches its destination—to prevent ransomware, malware, and more.

(32) Protect data using granular context-based policies: Zero trust policies verify access requests and rights based on context, including user identity, device, location, type of content, and the application being requested. Policies are adaptive, so user access privileges are continually reassessed as context changes.

(33) Reduce risk by eliminating the attack surface: With a zero trust approach, users connect directly to the apps and resources they need, never to networks (see ZTNA). Direct user-to-app and app-to-app connections eliminate the risk of lateral movement and prevent compromised devices from infecting other resources. Plus, users and apps are invisible to the internet, so they cannot be discovered or attacked.

(34) FIG. **1**C is a logical diagram illustrating zero trust policies with the cloud-based system **100** and a comparison with the conventional firewall-based approach. Zero trust with the cloud-based system **100** allows per session policy decisions and enforcement regardless of the user **102** location. Unlike the conventional firewall-based approach, this eliminates attack surfaces, there are no inbound connections; prevents lateral movement, the user is not on the network; prevents compromise, allowing encrypted inspection; and prevents data loss with inline inspection.

Example Implementation of the Cloud-Based System

(35) FIG. **2** is a network diagram of an example implementation of the cloud-based system **100**. In an embodiment, the cloud-based system **100** includes a plurality of enforcement nodes (EN) **150**, labeled as enforcement nodes **150-1**, **150-2**, **150**-N, interconnected to one another and interconnected to a central authority (CA) **152**. The nodes **150** and the central authority **152**, while described as nodes, can include one or more servers, including physical servers, virtual machines (VM) executed on physical hardware, etc. An example of a server is illustrated in FIG. **4**. The cloud-based system **100** further includes a log router **154** that connects to a storage cluster **156** for supporting log maintenance from the enforcement nodes **150**. The central authority **152** provide centralized policy, real-time threat updates, etc. and coordinates the distribution of this data between the enforcement nodes **150**. The enforcement nodes **150** provide an onramp to the users **102** and are configured to execute policy, based on the central authority **152**, for each user **102**. The enforcement nodes **150** can be geographically distributed, and the policy for each user **102** follows that user **102** as he or she connects to the nearest (or other criteria) enforcement node **150**.

(36) Of note, the cloud-based system **100** is an external system meaning it is separate from tenant's private networks (enterprise networks) as well as from networks associated with the devices **110**, **116**, and locations **112**, **118**. Also, of note, the present disclosure describes a private enforcement node **150**P that is both part of the cloud-based system **100** and part of a private network. Further, of note, the enforcement node described herein may simply be referred to as a node or cloud node. Also, the terminology enforcement node **150** is used in the context of the cloud-based system **100** providing cloud-based security. In the context of secure, private application access, the enforcement node **150** can also be referred to as a service edge or service edge node. Also, a service edge node **150** can be a public service edge node (part of the cloud-based system **100**) separate from an enterprise network or a private service edge node (still part of the cloud-based system **100**) but hosted either within an enterprise network, in a data center **114**, in a branch office **118**, etc. Further, the term nodes as used herein with respect to the cloud-based system **100** (including enforcement nodes, service edge nodes, etc.) can be one or more servers, including physical servers, virtual machines (VM) executed on physical hardware, etc., as described above. The service edge node **150** can also be a Secure Access Service Edge (SASE).

(37) The enforcement nodes **150** are full-featured secure internet gateways that provide integrated internet security. They inspect all web traffic bi-directionally for malware and enforce security, compliance, and firewall policies, as described herein, as well as various additional functionality. In an embodiment, each enforcement node **150** has two main modules for inspecting traffic and applying policies: a web module and a firewall module. The enforcement nodes **150** are deployed around the world and can handle hundreds of thousands of concurrent users with millions of concurrent sessions. Because of this, regardless of where the users **102** are, they can access the Internet **104** from any device, and the enforcement nodes **150** protect the traffic and apply corporate policies. The enforcement nodes **150** can implement various inspection engines therein, and optionally, send sandboxing to another system. The enforcement nodes **150** include significant fault tolerance capabilities, such as deployment in active-active mode to ensure availability and redundancy as well as continuous monitoring.

(38) In an embodiment, customer traffic is not passed to any other component within the cloud-based system **100**, and the enforcement nodes **150** can be configured never to store any data to disk. Packet data is held in memory for inspection and then, based on policy, is either forwarded or dropped. Log data generated for every transaction is compressed, tokenized, and exported over secure Transport Layer Security (TLS) connections to the log routers **154** that direct the logs to the storage cluster **156**, hosted in the appropriate geographical region, for each organization. In an embodiment, all data destined for or received from the Internet is processed through one of the enforcement nodes **150**. In another embodiment, specific data specified by each tenant, e.g., only email, only executable files, etc., is processed through one of the enforcement nodes **150**.

(39) Each of the enforcement nodes **150** may generate a decision vector D=[d1, d2, . . . , dn] for a content item of one or more parts C=[c1, c2, . . . , cm]. Each decision vector may identify a threat classification, e.g., clean, spyware, malware, undesirable content, innocuous, spam email, unknown, etc. For example, the output of each element of the decision vector D may be based on the output of one or more data inspection engines. In an embodiment, the threat classification may be reduced to a subset of categories, e.g., violating, non-violating, neutral, unknown. Based on the subset classification, the enforcement node **150** may allow the distribution of the content item, preclude distribution of the content item, allow distribution of the content item after a cleaning process, or perform threat detection on the content item. In an embodiment, the actions taken by one of the enforcement nodes **150** may be determinative on the threat classification of the content item and on a security policy of the tenant to which the content item is being sent from or from which the content item is being requested by. A content item is violating if, for any part C=[c1, c2, . . . , cm] of the content item, at any of the enforcement nodes **150**, any one of the data inspection engines generates an output that results in a classification of “violating.”
(40) The central authority **152** hosts all customer (tenant) policy and configuration settings. It monitors the cloud and provides a central location for software and database updates and threat intelligence. Given the multi-tenant architecture, the central authority **152** is redundant and backed up in multiple different data centers. The enforcement nodes **150** establish persistent connections to the central authority **152** to download all policy configurations. When a new user connects to an enforcement node **150**, a policy request is sent to the central authority **152** through this connection. The central authority **152** then calculates the policies that apply to that user **102** and sends the policy to the enforcement node **150** as a highly compressed bitmap.
(41) The policy can be tenant-specific and can include access privileges for users, websites and/or content that is disallowed, restricted domains, DLP dictionaries, etc. Once downloaded, a tenant's policy is cached until a policy change is made in the management system **120**. The policy can be tenant-specific and can include access privileges for users, websites and/or content that is disallowed, restricted domains, DLP dictionaries, etc. When this happens, all of the cached policies are purged, and the enforcement nodes **150** request the new policy when the user **102** next makes a request. In an embodiment, the enforcement node **150** exchange “heartbeats” periodically, so all enforcement nodes **150** are informed when there is a policy change. Any enforcement node **150** can then pull the change in policy when it sees a new request.
(42) The cloud-based system **100** can be a private cloud, a public cloud, a combination of a private cloud and a public cloud (hybrid cloud), or the like. Cloud computing systems and methods abstract away physical servers, storage, networking, etc., and instead offer these as on-demand and elastic resources. The National Institute of Standards and Technology (NIST) provides a concise and specific definition which states cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. Cloud computing differs from the classic client-server model by providing applications from a server that are executed and managed by a client's web browser or the like, with no installed client version of an application required. Centralization gives cloud service providers complete control over the versions of the browser-based and other applications provided to clients, which removes the need for version upgrades or license management on individual client computing devices. The phrase “Software as a Service” (SaaS) is sometimes used to describe application programs offered through cloud computing. A common shorthand for a provided cloud computing service (or even an aggregation of all existing cloud services) is “the cloud.” The cloud-based system **100** is illustrated herein as an example embodiment of a cloud-based system, and other implementations are also contemplated.
(43) As described herein, the terms cloud services and cloud applications may be used interchangeably. The cloud service **106** is any service made available to users on-demand via the

Internet, as opposed to being provided from a company's on-premises servers. A cloud application, or cloud app, is a software program where cloud-based and local components work together. The cloud-based system **100** can be utilized to provide example cloud services, including Zscaler Internet Access (ZIA), Zscaler Private Access (ZPA), and Zscaler Digital Experience (ZDX), all from Zscaler, Inc. (the assignee and applicant of the present application). Also, there can be multiple different cloud-based systems **100**, including ones with different architectures and multiple cloud services. The ZIA service can provide the access control, threat prevention, and data protection described above with reference to the cloud-based system **100**. ZPA can include access control, microservice segmentation, etc. The ZDX service can provide monitoring of user experience, e.g., Quality of Experience (QoE), Quality of Service (QOS), etc., in a manner that can gain insights based on continuous, inline monitoring. For example, the ZIA service can provide a user with Internet Access, and the ZPA service can provide a user with access to enterprise resources instead of traditional Virtual Private Networks (VPNs), namely ZPA provides Zero Trust Network Access (ZTNA). Those of ordinary skill in the art will recognize various other types of cloud services **106** are also contemplated. Also, other types of cloud architectures are also contemplated, with the cloud-based system **100** presented for illustration purposes.

Example Server Architecture

(44) FIG. **3** is a block diagram of a server **200**, which may be used in the cloud-based system **100**, in other systems, or standalone. For example, the enforcement nodes **150** and the central authority **152** may be formed as one or more of the servers **200**. The server **200** may be a digital computer that, in terms of hardware architecture, generally includes a processor **202**, input/output (I/O) interfaces **204**, a network interface **206**, a data store **208**, and memory **210**. It should be appreciated by those of ordinary skill in the art that FIG. **3** depicts the server **200** in an oversimplified manner, and a practical embodiment may include additional components and suitably configured processing logic to support known or conventional operating features that are not described in detail herein. The components (**202**, **204**, **206**, **208**, and **210**) are communicatively coupled via a local interface **212**. The local interface **212** may be, for example, but not limited to, one or more buses or other wired or wireless connections, as is known in the art. The local interface **212** may have additional elements, which are omitted for simplicity, such as controllers, buffers (caches), drivers, repeaters, and receivers, among many others, to enable communications. Further, the local interface **212** may include address, control, and/or data connections to enable appropriate communications among the aforementioned components.

(45) The processor **202** is a hardware device for executing software instructions. The processor **202** may be any custom made or commercially available processor, a Central Processing Unit (CPU), an auxiliary processor among several processors associated with the server **200**, a semiconductor-based microprocessor (in the form of a microchip or chipset), or generally any device for executing software instructions. When the server **200** is in operation, the processor **202** is configured to execute software stored within the memory **210**, to communicate data to and from the memory **210**, and to generally control operations of the server **200** pursuant to the software instructions. The I/O interfaces **204** may be used to receive user input from and/or for providing system output to one or more devices or components.

(46) The network interface **206** may be used to enable the server **200** to communicate on a network, such as the Internet **104**. The network interface **206** may include, for example, an Ethernet card or adapter or a Wireless Local Area Network (WLAN) card or adapter. The network interface **206** may include address, control, and/or data connections to enable appropriate communications on the network. A data store **208** may be used to store data. The data store **208** may include any of volatile memory elements (e.g., random access memory (RAM, such as DRAM, SRAM, SDRAM, and the like)), nonvolatile memory elements (e.g., ROM, hard drive, tape, CDROM, and the like), and combinations thereof.

(47) Moreover, the data store **208** may incorporate electronic, magnetic, optical, and/or other types

of storage media. In one example, the data store **208** may be located internal to the server **200**, such as, for example, an internal hard drive connected to the local interface **212** in the server **200**. Additionally, in another embodiment, the data store **208** may be located external to the server **200** such as, for example, an external hard drive connected to the I/O interfaces **204** (e.g., SCSI or USB connection). In a further embodiment, the data store **208** may be connected to the server **200** through a network, such as, for example, a network-attached file server.

(48) The memory **210** may include any of volatile memory elements (e.g., random access memory (RAM, such as DRAM, SRAM, SDRAM, etc.)), nonvolatile memory elements (e.g., ROM, hard drive, tape, CDROM, etc.), and combinations thereof. Moreover, the memory **210** may incorporate electronic, magnetic, optical, and/or other types of storage media. Note that the memory **210** may have a distributed architecture, where various components are situated remotely from one another but can be accessed by the processor **202**. The software in memory **210** may include one or more software programs, each of which includes an ordered listing of executable instructions for implementing logical functions. The software in the memory **210** includes a suitable Operating System (O/S) **214** and one or more programs **216**. The operating system **214** essentially controls the execution of other computer programs, such as the one or more programs **216**, and provides scheduling, input-output control, file and data management, memory management, and communication control and related services. The one or more programs **216** may be configured to implement the various processes, algorithms, methods, techniques, etc. described herein.

Example User Device Architecture

(49) FIG. **4** is a block diagram of a user device **300**, which may be used with the cloud-based system **100** or the like. Specifically, the user device **300** can form a device used by one of the users **102**, and this may include common devices such as laptops, smartphones, tablets, netbooks, personal digital assistants, MP3 players, cell phones, e-book readers, IoT devices, servers, desktops, printers, televisions, streaming media devices, and the like. The user device **300** can be a digital device that, in terms of hardware architecture, generally includes a processor **302**, I/O interfaces **304**, a network interface **306**, a data store **308**, and memory **310**. It should be appreciated by those of ordinary skill in the art that FIG. **4** depicts the user device **300** in an oversimplified manner, and a practical embodiment may include additional components and suitably configured processing logic to support known or conventional operating features that are not described in detail herein. The components (**302**, **304**, **306**, **308**, and **302**) are communicatively coupled via a local interface **312**. The local interface **312** can be, for example, but not limited to, one or more buses or other wired or wireless connections, as is known in the art. The local interface **312** can have additional elements, which are omitted for simplicity, such as controllers, buffers (caches), drivers, repeaters, and receivers, among many others, to enable communications. Further, the local interface **312** may include address, control, and/or data connections to enable appropriate communications among the aforementioned components.

(50) The processor **302** is a hardware device for executing software instructions. The processor **302** can be any custom made or commercially available processor, a CPU, an auxiliary processor among several processors associated with the user device **300**, a semiconductor-based microprocessor (in the form of a microchip or chipset), or generally any device for executing software instructions. When the user device **300** is in operation, the processor **302** is configured to execute software stored within the memory **310**, to communicate data to and from the memory **310**, and to generally control operations of the user device **300** pursuant to the software instructions. In an embodiment, the processor **302** may include a mobile optimized processor such as optimized for power consumption and mobile applications. The I/O interfaces **304** can be used to receive user input from and/or for providing system output. User input can be provided via, for example, a keypad, a touch screen, a scroll ball, a scroll bar, buttons, a barcode scanner, and the like. System output can be provided via a display device such as a Liquid Crystal Display (LCD), touch screen, and the like.

(51) The network interface **306** enables wireless communication to an external access device or network. Any number of suitable wireless data communication protocols, techniques, or methodologies can be supported by the network interface **306**, including any protocols for wireless communication. The data store **308** may be used to store data. The data store **308** may include any of volatile memory elements (e.g., random access memory (RAM, such as DRAM, SRAM, SDRAM, and the like)), nonvolatile memory elements (e.g., ROM, hard drive, tape, CDROM, and the like), and combinations thereof. Moreover, the data store **308** may incorporate electronic, magnetic, optical, and/or other types of storage media.
(52) The memory **310** may include any of volatile memory elements (e.g., random access memory (RAM, such as DRAM, SRAM, SDRAM, etc.)), nonvolatile memory elements (e.g., ROM, hard drive, etc.), and combinations thereof. Moreover, the memory **310** may incorporate electronic, magnetic, optical, and/or other types of storage media. Note that the memory **310** may have a distributed architecture, where various components are situated remotely from one another but can be accessed by the processor **302**. The software in memory **310** can include one or more software programs, each of which includes an ordered listing of executable instructions for implementing logical functions. In the example of FIG. **3**, the software in the memory **310** includes a suitable operating system **314** and programs **316**. The operating system **314** essentially controls the execution of other computer programs and provides scheduling, input-output control, file and data management, memory management, and communication control and related services. The programs **316** may include various applications, add-ons, etc. configured to provide end user functionality with the user device **300**. For example, example programs **316** may include, but not limited to, a web browser, social networking applications, streaming media applications, games, mapping and location applications, electronic mail applications, financial applications, and the like. In a typical example, the end-user typically uses one or more of the programs **316** along with a network such as the cloud-based system **100**.
(53) Zero Trust Network Access Using the Cloud-Based System
(54) FIG. **5** is a network diagram of a Zero Trust Network Access (ZTNA) application utilizing the cloud-based system **100**. For ZTNA, the cloud-based system **100** can dynamically create a connection through a secure tunnel between an endpoint (e.g., users **102**A, **102**B) that are remote and an on-premises connector **400** that is either located in cloud file shares and applications **402** and/or in an enterprise network **410** that includes enterprise file shares and applications **404**. The connection between the cloud-based system **100** and on-premises connector **400** is dynamic, on-demand, and orchestrated by the cloud-based system **100**. A key feature is its security at the edge—there is no need to punch any holes in the existing on-premises firewall. The connector **400** inside the enterprise (on-premises) “dials out” and connects to the cloud-based system **100** as if too were an endpoint. This on-demand dial-out capability and tunneling authenticated traffic back to the enterprise is a key differentiator for ZTNA. Also, this functionality can be implemented in part by an application **350** on the user device **300**. Also, the applications **402**, **404** can include B2B applications. Note, the difference between the applications **402**, **404** is the applications **402** are hosted in the cloud, whereas the applications **404** are hosted on the enterprise network **410**. The services described herein contemplates use with either or both of the applications **402**, **404**.
(55) The paradigm of virtual private access systems and methods is to give users network access to get to an application and/or file share, not to the entire network. If a user is not authorized to get the application, the user should not be able even to see that it exists, much less access it. The virtual private access systems and methods provide an approach to deliver secure access by decoupling applications **402**, **404** from the network, instead of providing access with a connector **400**, in front of the applications **402**, **404**, an application on the user device **300**, a central authority **152** to push policy, and the cloud-based system **100** to stitch the applications **402**, **404** and the software connectors **400** together, on a per-user, per-application basis.
(56) With the virtual private access, users can only see the specific applications **402**, **404** allowed

by the central authority **152**. Everything else is “invisible” or “dark” to them. Because the virtual private access separates the application from the network, the physical location of the application **402**, **404** becomes irrelevant—if applications **402**, **404** are located in more than one place, the user is automatically directed to the instance that will give them the best performance. The virtual private access also dramatically reduces configuration complexity, such as policies/firewalls in the data centers. Enterprises can, for example, move applications to Amazon Web Services or Microsoft Azure, and take advantage of the elasticity of the cloud, making private, internal applications behave just like the marketing leading enterprise applications. Advantageously, there is no hardware to buy or deploy because the virtual private access is a service offering to end-users and enterprises.

(57) § 6.0 Digital Experience Monitoring

(58) FIG. **6** is a network diagram of the cloud-based system **100** in an application of digital experience monitoring. Here, the cloud-based system **100** providing security as a service as well as ZTNA, can also be used to provide real-time, continuous digital experience monitoring, as opposed to conventional approaches (synthetic probes). A key aspect of the architecture of the cloud-based system **100** is the inline monitoring. This means data is accessible in real-time for individual users from end-to-end. As described herein, digital experience monitoring can include monitoring, analyzing, and improving the digital user experience.

(59) The cloud-based system **100** connects users **102** at the locations **110**, **112**, **118** to the applications **402**, **404**, the Internet **104**, the cloud services **106**, etc. The inline, end-to-end visibility of all users enables digital experience monitoring. The cloud-based system **100** can monitor, diagnose, generate alerts, and perform remedial actions with respect to network endpoints, network components, network links, etc. The network endpoints can include servers, virtual machines, containers, storage systems, or anything with an IP address, including the Internet of Things (IoT), cloud, and wireless endpoints. With these components, these network endpoints can be monitored directly in combination with a network perspective. Thus, the cloud-based system **100** provides a unique architecture that can enable digital experience monitoring, network application monitoring, infrastructure component interactions, etc. Of note, these various monitoring aspects require no additional components—the cloud-based system **100** leverages the existing infrastructure to provide this service.

(60) Again, digital experience monitoring includes the capture of data about how end-to-end application availability, latency, and quality appear to the end user from a network perspective. This is limited to the network traffic visibility and not within components, such as what application performance monitoring can accomplish. Networked application monitoring provides the speed and overall quality of networked application delivery to the user in support of key business activities. Infrastructure component interactions include a focus on infrastructure components as they interact via the network, as well as the network delivery of services or applications. This includes the ability to provide network path analytics.

(61) The cloud-based system **100** can enable real-time performance and behaviors for troubleshooting in the current state of the environment, historical performance and behaviors to understand what occurred or what is trending over time, predictive behaviors by leveraging analytics technologies to distill and create actionable items from the large dataset collected across the various data sources, and the like. The cloud-based system **100** includes the ability to directly ingest any of the following data sources network device-generated health data, network device-generated traffic data, including flow-based data sources inclusive of NetFlow and IPFIX, raw network packet analysis to identify application types and performance characteristics, HTTP request metrics, etc. The cloud-based system **100** can operate at 10 gigabits (10 G) Ethernet and higher at full line rate and support a rate of 100,000 or more flows per second or higher.

(62) The applications **402**, **404** can include enterprise applications, Office **365**, Salesforce, Skype, Google apps, internal applications, etc. These are critical business applications where user

experience is important. The objective here is to collect various data points so that user experience can be quantified for a particular user, at a particular time, for purposes of analyzing the experience as well as improving the experience. In an embodiment, the monitored data can be from different categories, including application-related, network-related, device-related (also can be referred to as endpoint-related), protocol-related, etc. Data can be collected at the application **350** or the cloud edge to quantify user experience for specific applications, i.e., the application-related and device-related data. The cloud-based system **100** can further collect the network-related and the protocol-related data (e.g., Domain Name System (DNS) response time).

(63) Application-Related Data

(64) TABLE-US-00001 Page Load Time Redirect count (#) Page Response Time Throughput (bps) Document Object Model Total size (bytes) (DOM) Load Time Total Downloaded bytes Page error count (#) App availability (%) Page element count by category (#)

Network-Related Data

(65) TABLE-US-00002 HTTP Request metrics Bandwidth Server response time Jitter Ping packet loss (%) Trace Route Ping round trip DNS lookup trace Packet loss (%) GRE/IPSec tunnel monitoring Latency MTU and bandwidth measurements

Device-Related Data (Endpoint-Related Data)

(66) TABLE-US-00003 System details Network (config) Central Processing Unit (CPU) Disk Memory (RAM) Processes Network (interfaces) Applications

(67) Metrics could be combined. For example, device health can be based on a combination of CPU, memory, etc. Network health could be a combination of Wi-Fi/LAN connection health, latency, etc. Application health could be a combination of response time, page loads, etc. The cloud-based system **100** can generate service health as a combination of CPU, memory, and the load time of the service while processing a user's request. The network health could be based on the number of network path(s), latency, packet loss, etc.

(68) The lightweight connector **400** can also generate similar metrics for the applications **402**, **404**. In an embodiment, the metrics can be collected while a user is accessing specific applications that user experience is desired for monitoring. In another embodiment, the metrics can be enriched by triggering synthetic measurements in the context of an inline transaction by the application **350** or cloud edge. The metrics can be tagged with metadata (user, time, app, etc.) and sent to a logging and analytics service for aggregation, analysis, and reporting. Further, network administrators can get UEX reports from the cloud-based system **100**. Due to the inline nature and the fact the cloud-based system **100** is an overlay (in-between users and services/applications), the cloud-based system **100** enables the ability to capture user experience metric data continuously and to log such data historically. As such, a network administrator can have a long-term detailed view of the network and associated user experience.

(69) Zero Trust Lensing

(70) Again, the present disclosure relates to systems and methods for zero trust lensing of cloud-based traffic. Zero trust lensing is a method by which client-initiated traffic is intercepted and filtered through an alternate viewing mechanism prior to reaching its destination. In various embodiments, neither the source nor destination applications associated with the traffic are aware of the translation and sit independent and unaltered during the exchange.

(71) Present systems and methods are analogous to using lenses (glasses/contacts) to alter the experience of a wearer with neither the wearer's eyes, nor the viewed object being altered in any way. Lenses within a zero trust ecosystem allow for interchangeable consumption of destination services or workloads. This consumption can be altered as necessary to suit various requirements represented in the various embodiments disclosed herein. An example would be allowing an initiator, using a modern browser, to access a legacy web service that wouldn't otherwise be rendered in a modern browser.

(72) FIG. **7** is a flow diagram of an embodiment for zero trust lensing of the present disclosure.

Various embodiments of zero trust lensing use containers and/or virtual machines positioned between the initiating client (user) **702** and the destination application **704**, or any other destination workload, to alter the experience for the user **702** to best consume the destination application **704**. The lens or set of lenses **710** can be a collection of rendering engines, henceforth defined as lens engines, stored and called from. These lens engines include but are not limited to Application Programming Interfaces (APIs), containers, libraries, virtual machine hosted services, and the like.
(73) At runtime, a lens can be leveraged to alter the experience and functionality as traffic passes between an initiating application client **702** and the application destination **704**. The lens layer **712** can be substituted as desired to provide an altered experience as traffic passes between the client **702** and destination **704**. The present disclosure is aimed at a range of use cases primarily but not limited to legacy browser or application, app streaming, and a full Virtual Desktop Infrastructure (VDI) experience. In various embodiments, the lenses can act as a means of providing legacy or future functionality and viewing experiences without altering the client or the destination. The present lens solution is initially designed to offer the service through a rendered web based application, but could be extended to offer additional service such as, but not limited to, hosted application orchestration.
(74) Traditional user to application exchange includes a 1:2:1 relationship between the source (initiator) and the destination. Conventional exchange relies on network traffic initiated from a device client such as a web browser or purpose-built application over a network communication medium which is either private or public to a destination application. In such a relationship, irrespective of the communication medium (direct or indirect via proxy-based architecture) the functionality and experience are dictated by the capabilities of the client and the ability of the destination to receive and respond. This results in an experience that is anchored on the capabilities of the client and that of the destination. This is an example of a highly aligned and highly coupled relationship.
(75) When a client operating system or initiating application is altered (as in the case of version revisions) this often results in an incompatibility between source and destination. The relationship is inflexible and means that deviation of either the source or destination capabilities will result in an inability to function. Applications which cannot be updated to compensate for the misalignment often result in the destination application being labeled as legacy technology. Various traditional approaches to solving such cases are further described below.
(76) Various traditional responses include freezing both client and destination applications to avoid incompatibility. This results in a security risk as new threats cannot be remediated through updates. This response is also not sustainable as application vendors will typically end support. Another response includes running old and new client versions side by side. Consequences of this method include poor user experience with users required to switch between old and new versions. This includes security risks and is not sustainable due to vendor support or interoperability. Other responses include shifting the client application to a VDI environment. This results in Poor user experience with users required to switch between VDI and client hosted software. It also increases complicity and cost by needing to run a VDI environment while introducing security risks and is not sustainable due to vendor support or interoperability.
(77) The present zero trust lensing functionality is designed specifically to decouple the relationship between a client device initiating traffic and that of the receiving destination workload by hosting independently and in-line to a client request and a destination workload.
(78) The zero trust lensing service includes a number of parts and will augment existing zero trust functionality that could be integrated into existing services or delivered separately as a new service. Whilst lensing is complimentary to a zero trust architecture, the functionality should be seen as modular, thus could be operated independent of a zero trust exchange.
(79) A zero trust lens can be contemplated as a stored or called upon repository of the alternative rendering engine. It will reside in-line between a client and a destination application. Each lens can

be instantiated to deliver a specific purpose or function such as a specific application but can also be leveraged to provide a full VDI experience. As such, present embodiments include an architecture which can allow for the zero trust service to consume the lens engine, along with the zero trust design principals to provide additional separation to the zero trust lens architecture.
(80) A zero trust lens library **708** can be contemplated as a repository of APIs, containers, libraries, virtual machine hosted services, etc. The library represents an interchangeable set of images to provide specific functionality when instigated as a workload. A repository is positioned as image categories including but not limited to stock application lenses, custom application lenses, vendor application lens marketplace lenses, and VDI lenses.
(81) Stock application lenses are point in time crafted lenses with specific software versions created for a mainstream audience. This includes but is not be limited to specific web browsers, java, and software library versions. Lenses can be across multiple tenants. Examples of static lenses can be, but are not limited to, internet version 6, 7, 8, 9, Chrome, and Firefox.
(82) Custom application lenses are bespoke customer images designed to host customer proprietary and in house developed applications.
(83) Vendor application lens marketplace lenses include images created and maintained by mainstream vendors independent to that of the customer and or the cloud provider. An example can be, but is not limited to, a SAP client.
(84) VDI lenses include images containing full operating systems. Examples include Microsoft Windows, Linux (multiple distributions), MacOS, and the like.
(85) A zero trust lens marketplace is an extension to the zero trust lens library **708**. The marketplace represents a collection of vendor maintained and provided lens images which would incur additional and or separate licensing and usage agreements. The marketplace can facilitate a brokered transactional relationship between a customer and a lens vendor. Cloud providers can host the workload independent to that of the lens vendor and on behalf of a customer.
(86) Zero trust lens workloads **706** include running workloads instigated from a lens image. Workloads can represent a 1:2:1 relationship to that of an instigating client/request. Workloads are designed to be created and utilized for the life of a session and subsequently terminated at the end of each use. The zero trust lens broker represents the logical relationship between a lens, a user/client request and a workload definition. The policy broker is responsible for but not limited to 1) Identifying the lens image to be instigated 2) defining the properties of a lens (i.e., workload duration, workload quantities/scale up and scale down definitions, and workload grace periods). The workload engine is a controller which is responsible for 1) Workload instigation, 2) Workload termination, 3) Workload traffic management, 4) enforcement of policy broker properties, and such. The connection broker is responsible for regulating the connection between running workloads to the customer application destination.
(87) A tenant is defined as a dedicated runtime environment for hosting lens workloads. It includes a virtual network boundary (VNET or VPC) with pre provisioned hosts for running workloads. It also hosts an instance of a lens renderer as well as a connection broker to a destination workload using cloud connectors which will be bound to a customer private access tenant.
(88) FIG. **8** is a flow diagram of an embodiment of the present zero trust lens architecture. Again, various embodiments of the present zero trust lens systems and methods include lens policy services **802**, lens databases/marketplaces **708**, lens workload services **804**, and connection brokers **806**. In various embodiments, a client initiates connectivity to a customer application (destination workload) via a cloud provider client or web browser. Traffic is intercepted via the private access client or by Domain Name System (DNS) resolution as per browser access functionality. Authorization, context, and user entitlement are evaluated and enforced.
(89) Requests are passed to the zero trust lens engine to determine lens entitlement, and workload running properties. If a lens is required, the request is passed to the zero trust lens workload engine which will instantiate a workload based on the lens properties (running session). If a lens is not

required (traditional private access), requests will be passed to the private access connection broker as per a normal request. Once a workload session is running, user traffic is passed to the rendering provider. This is the rendering provider between the client and the zero trust lens workload. Running zero trust lens workloads will connect to a customer destination workload via an outbound cloud connector bound to a customer private access tenant. The connection broker will provide a Transport Layer Security (TLS) connection to a customer side running application. Once a customer session is terminated, the workload engine will terminate the running lens workload and perform any necessary maintenance.

(90) FIG. **9** is a flow diagram of an embodiment of zero trust lensing utilizing various side containers. A customer (user) **902** can request an application with legacy isolation requirements. Present systems intercept and rout the traffic to a rendering engine. The rendering provider facilitates an Hyper Text Markup Language 5 (HTML5) broker to either standard containers **904** or customer bespoke image containers **906** via Virtual Network Computing (VNC). Containers call customer dedicated cloud connectors for access to customer hosting locations **908** (application connector on the receiving side). Customer side applications or cloud connectors receive the requests and connect to internal resources (i.e., traditional private access exchange).

(91) FIG. **10** is a flow diagram of an embodiment of the present zero trust lens workflow logic. After receiving a request from a user, lens policies are validated, and responsive to no lens being necessary, the traffic is passed to the destination workload. If a lens is required, the lens availability is determined. If a lens is available, the traffic is passed to the lens renderer. If a lens is not available, a lens workload is instigated and the lens is again checked for availability, where upon availability of the lens, the traffic is passed to the lens renderer.

(92) The proposed systems and methods enable customers of the service to access legacy applications through modern browsers. Systems extend Business to Business (B2B) use cases for legacy and modern applications resulting in increased agility. Systems also extend the life of legacy application investments and future proof existing applications which will become future legacy applications.

(93) The systems and methods further isolate communication via an indirect access model which provides segmentation for legacy applications which reduces risk. Various embodiments further decouple the direct relationship to that of a desktop operating system so that customers do not need to utilize unsupported and end of life applications locally. This shifts compute power to zero trust lens hosting infrastructure resulting in extended life of current end user devices and better performance. Various embodiments of the present systems and methods further remove conflicts between cohosting modern and legacy applications side by side. Additionally, embodiments extend current private access functionality without additional software or client installations.

(94) FIG. **11** is a flow chart of a process **1100** for zero trust lensing of cloud-based traffic. The process **1100** includes intercepting a plurality of requests from one or more users in a cloud-based system, wherein the requests are for connectivity to one or more destination workloads (step **1102**); determining a request of the plurality of requests requires lensing (step **1104**); identifying an appropriate lens for rendering the request (step **1106**); and initiating and utilizing a lens workload for processing a session associated with the request, wherein the session is processed in-line between the user and the workload destination without the user nor the workload destination being aware of the lens (step **1108**).

(95) Process **1100** further includes wherein the lens is adapted to provide one of a specific function associated with a specific application and a full Virtual Desktop Infrastructure (VDI) experience. The steps are performed within a cloud-based system utilizing a zero trust architecture. The appropriate lens is selected from a repository including a plurality of lenses, and wherein the plurality of lenses includes any of Application Programing Interfaces (APIs), containers, libraries, and virtual machine hosted services. The repository of lenses includes any of stock application lenses, custom application lenses, lenses from a vendor application lens marketplace, and Virtual

Desktop Infrastructure (VDI) lenses. The vendor application lens marketplace is an extension of the lens repository, and wherein the vendor application lens marketplace includes a collection of vendor maintained and provisioned lenses. Lens workloads are created and utilized for the life of a session and subsequently terminated at the end of each session. The request can be passed to a private access connector responsive to determining that no lens is required.

CONCLUSION

(96) It will be appreciated that some embodiments described herein may include one or more generic or specialized processors ("one or more processors") such as microprocessors; Central Processing Units (CPUs); Digital Signal Processors (DSPs): customized processors such as Network Processors (NPs) or Network Processing Units (NPUs), Graphics Processing Units (GPUs), or the like; Field Programmable Gate Arrays (FPGAs); and the like along with unique stored program instructions (including both software and firmware) for control thereof to implement, in conjunction with certain non-processor circuits, some, most, or all of the functions of the methods and/or systems described herein. Alternatively, some or all functions may be implemented by a state machine that has no stored program instructions, or in one or more Application Specific Integrated Circuits (ASICs), in which each function or some combinations of certain of the functions are implemented as custom logic or circuitry. Of course, a combination of the aforementioned approaches may be used. For some of the embodiments described herein, a corresponding device such as hardware, software, firmware, and a combination thereof can be referred to as "circuitry configured or adapted to," "logic configured or adapted to," etc. perform a set of operations, steps, methods, processes, algorithms, functions, techniques, etc. as described herein for the various embodiments.

(97) Moreover, some embodiments may include a non-transitory computer-readable storage medium having computer readable code stored thereon for programming a computer, server, appliance, device, processor, circuit, etc. each of which may include a processor to perform functions as described and claimed herein. Examples of such computer-readable storage mediums include, but are not limited to, a hard disk, an optical storage device, a magnetic storage device, a ROM (Read Only Memory), a PROM (Programmable Read Only Memory), an EPROM (Erasable Programmable Read Only Memory), an EEPROM (Electrically Erasable Programmable Read Only Memory), Flash memory, and the like. When stored in the non-transitory computer readable medium, software can include instructions executable by a processor or device (e.g., any type of programmable circuitry or logic) that, in response to such execution, cause a processor or the device to perform a set of operations, steps, methods, processes, algorithms, functions, techniques, etc. as described herein for the various embodiments.

(98) Although the present disclosure has been illustrated and described herein with reference to preferred embodiments and specific examples thereof, it will be readily apparent to those of ordinary skill in the art that other embodiments and examples may perform similar functions and/or achieve like results. All such equivalent embodiments and examples are within the spirit and scope of the present disclosure, are contemplated thereby, and are intended to be covered by the following claims. The foregoing sections include headers for various embodiments and those skilled in the art will appreciate these various embodiments may be used in combination with one another as well as individually.

Claims

1. A method implemented within a cloud-based system utilizing a zero trust architecture that enforces session-based isolation and prevents direct exposure of users and destination workloads, the method comprising steps of: intercepting a plurality of requests from one or more users in the cloud-based system, wherein the requests are for connectivity to one or more destination workloads; determining a request of the plurality of requests requires lensing; identifying an

appropriate lens for rendering the request; and initiating a transient lens workload as a runtime executable instance selected from a repository of lenses, and utilizing the transient lens workload for processing a session associated with the request, wherein the session is processed in-line between the user and the workload destination through the transient lens workload, such that the session traffic is mediated without the user nor the workload destination being aware of the lens, and wherein lens workloads are instantiated at session initiation and subsequently terminated upon session conclusion.

2. The method of claim 1, wherein the lens is adapted to provide one of a specific function associated with a specific application and a full Virtual Desktop Infrastructure (VDI) experience.

3. The method of claim 1, wherein the appropriate lens is selected from a repository comprising a plurality of lenses, and wherein the plurality of lenses includes any of Application Programing Interfaces (APIs), containers, libraries, and virtual machine hosted services.

4. The method of claim 3, wherein the repository of lenses includes any of stock application lenses, custom application lenses, lenses from a vendor application lens marketplace, and Virtual Desktop Infrastructure (VDI) lenses.

5. The method of claim 4, wherein the vendor application lens marketplace is an extension of the lens repository, and wherein the vendor application lens marketplace includes a collection of vendor maintained and provisioned lenses.

6. The method of claim 1, wherein the request is passed to a private access connector responsive to determining that no lens is required.

7. A non-transitory computer-readable medium comprising instructions that, when executed, cause one or more processors associated with a cloud-based system utilizing a zero trust architecture that enforces session-based isolation and prevents direct exposure of users and destination workloads to perform steps of: intercepting a plurality of requests from one or more users in the cloud-based system, wherein the requests are for connectivity to one or more destination workloads; determining a request of the plurality of requests requires lensing; identifying an appropriate lens for rendering the request; and initiating a transient lens workload as a runtime executable instance selected from a repository of lenses, and utilizing the transient lens workload for processing a session associated with the request, wherein the session is processed in-line between the user and the workload destination through the transient lens workload, such that the session traffic is mediated without the user nor the workload destination being aware of the lens, and wherein lens workloads are instantiated at session initiation and subsequently terminated upon session conclusion.

8. The non-transitory computer-readable medium of claim 7, wherein the lens is adapted to provide one of a specific function associated with a specific application and a full Virtual Desktop Infrastructure (VDI) experience.

9. The non-transitory computer-readable medium of claim 7, wherein the appropriate lens is selected from a repository comprising a plurality of lenses, and wherein the plurality of lenses includes any of Application Programing Interfaces (APIs), containers, libraries, and virtual machine hosted services.

10. The non-transitory computer-readable medium of claim 9, wherein the repository of lenses includes any of stock application lenses, custom application lenses, lenses from a vendor application lens marketplace, and Virtual Desktop Infrastructure (VDI) lenses.

11. The non-transitory computer-readable medium of claim 10, wherein the vendor application lens marketplace is an extension of the lens repository, and wherein the vendor application lens marketplace includes a collection of vendor maintained and provisioned lenses.

12. The non-transitory computer-readable medium of claim 7, wherein the request is passed to a private access connector responsive to determining that no lens is required.

13. A cloud-based system utilizing a zero trust architecture that enforces session-based isolation and prevents direct exposure of users and destination workloads comprising: one or more

processors and memory storing instructions that, when executed, cause the one or more processors to: intercept a plurality of requests from one or more users in the cloud-based system, wherein the requests are for connectivity to one or more destination workloads; determine a request of the plurality of requests requires lensing; identify an appropriate lens for rendering the request; and initiate a transient lens workload as a runtime executable instance selected from a repository of lenses, and utilize the transient lens workload for processing a session associated with the request, wherein the session is processed in-line between the user and the workload destination through the transient lens workload, such that the session traffic is mediated without the user nor the workload destination being aware of the lens, and wherein lens workloads are instantiated at session initiation and subsequently terminated upon session conclusion.

14. The cloud-based system of claim 13, wherein the lens is adapted to provide one of a specific function associated with a specific application and a full Virtual Desktop Infrastructure (VDI) experience.

15. The cloud-based system of claim 13, wherein the appropriate lens is selected from a repository comprising a plurality of lenses, and wherein the plurality of lenses includes any of Application Programing Interfaces (APIs), containers, libraries, and virtual machine hosted services.